

Lab 3: XML, XML Schema, JSON, AJAX and Fetch API

Introduction

Modern web applications constantly exchange information between browsers and servers.

Examples include

- Online banking
- Social media
- E-commerce websites
- Student portals
- Online examination systems

The browser and server exchange information using standard data formats.

Earlier web applications used **XML**, whereas modern applications predominantly use **JSON**. Data exchange is performed asynchronously using technologies such as **AJAX** and the **Fetch API**, allowing web pages to update content without requiring a full page refresh.

The technologies covered in this lab are summarized below.

Technology	Purpose
XML	Store and transport structured data
XML Schema (XSD)	Validate XML documents
JSON	Lightweight data interchange format
AJAX	Asynchronous communication using XMLHttpRequest
Fetch API	Modern API for asynchronous communication

PART A – XML

What is XML?

XML (Extensible Markup Language) is a markup language used to store and transport structured information.

Unlike HTML, XML focuses on describing **data**, not how it should be displayed.

Typical applications include

- Configuration files
- RSS feeds
- Office documents
- SOAP Web Services
- Data interchange between systems

Characteristics of XML

- Extensible
- Platform independent
- Human readable
- Self-descriptive
- Supports validation

XML Syntax Rules

- Exactly one root element.
- Tags are case sensitive.
- Every opening tag must have a matching closing tag.
- Elements must be properly nested.
- Attribute values must be enclosed in quotation marks.

Example XML Document

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<student>
```

```
  <registerNumber>CB22001</registerNumber>
```

```
  <name>Alice</name>
```

```
  <department>CSE</department>
```

```
<semester>5</semester>
```

```
<cgpa>8.92</cgpa>
```

```
</student>
```

Code Explanation

- `<?xml ...?>` specifies the XML version and encoding.
- `<student>` is the root element.
- Child elements store the student's details.
- The hierarchical structure makes the document easy to parse.

Activity 1

Create an XML document containing the details of five students.

PART B – XML Schema (XSD)

Why XML Schema?

Although XML defines the structure of a document, it cannot ensure that the data is valid.

Example:

```
<semester>Five</semester>
```

This XML is well-formed but semantically incorrect if the semester must be numeric.

XML Schema (XSD) allows developers to define the expected structure and data types of XML documents.

Benefits

- Enforces data types.
- Defines mandatory elements.

- Specifies element order.
- Restricts valid values.
- Improves data integrity.

XML Schema Example

```
<?xml version="1.0"?>

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="student">

    <xs:complexType>

      <xs:sequence>

        <xs:element name="registerNumber" type="xs:string"/>

        <xs:element name="name" type="xs:string"/>

        <xs:element name="department" type="xs:string"/>

        <xs:element name="semester" type="xs:integer"/>

        <xs:element name="cgpa" type="xs:decimal"/>

      </xs:sequence>

    </xs:complexType>

  </xs:element>

</xs:schema>
```

Code Explanation

- xs:schema defines the schema.
- xs:sequence enforces the order of elements.

- xs:integer ensures only integer values are accepted.
- xs:decimal restricts values to decimal numbers.

Activity 2

Modify the schema to ensure:

- Semester is between 1 and 8.
- CGPA is between 0 and 10.

PART C – JSON

Introduction

JSON (JavaScript Object Notation) is a lightweight, text-based data format used for exchanging information between clients and servers.

Today, nearly all REST APIs exchange data using JSON because it is compact and easy to process.

JSON Objects

```
{  
  "name": "Alice",  
  "department": "CSE",  
  "cgpa": 9.1  
}
```

JSON Arrays

```
[  
  {  
    "name": "Alice"  
  },  
  {  
    "name": "Bob"  
  }  
]
```

```
}  
]
```

Why JSON?

- Smaller than XML.
- Easy to read.
- Native support in JavaScript.
- Faster to parse.
- Widely used in REST APIs.

Accessing JSON

```
let student = {  
  name: "Alice",  
  cgpa: 9.2  
};
```

```
console.log(student.name);
```

Activity 3

Create a JSON array containing five students.

XML vs JSON

XML	JSON
Tag based	Key-value based
Verbose	Compact
Larger file size	Smaller file size
Slower parsing	Faster parsing
Supports schema validation	No built-in schema
Human readable	Human readable

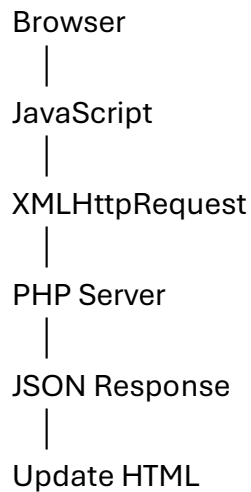
PART D – AJAX

Introduction

AJAX (Asynchronous JavaScript and XML) is a technique that enables web pages to communicate with the server **without reloading the entire page**.

Although AJAX originally used XML, modern applications typically exchange JSON data.

AJAX Workflow



PHP Backend

```
<?php
header("Content-Type: application/json");
```

```
$data = [
    "name"=>"Alice",
    "cgpa"=>9.2
];
```

```
echo json_encode($data);
?>
```

AJAX Example

```
let xhr = new XMLHttpRequest();

xhr.open("GET","students.php",true);

xhr.onload = function(){

    let student = JSON.parse(xhr.responseText);

    document.getElementById("result").innerHTML =
        student.name + " " + student.cgpa;

};

xhr.send();
```

Code Explanation

- XMLHttpRequest() creates a request object.
- open() configures the request.
- send() sends the request.
- JSON.parse() converts the JSON response into a JavaScript object.
- The webpage is updated dynamically without refreshing.

Activity 4

Modify the PHP script to return an array of students and display them in an HTML table.

PART E – Fetch API

Introduction

The Fetch API is the modern replacement for XMLHttpRequest. It provides a cleaner, Promise-based interface for sending HTTP requests and processing server responses asynchronously.

It is widely used in modern web applications to communicate with RESTful APIs and retrieve data without reloading the webpage.

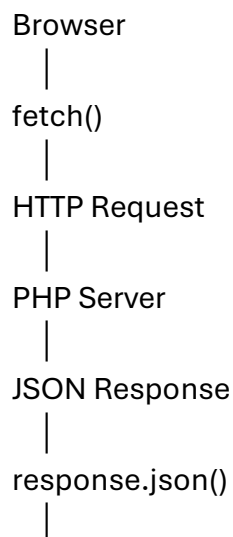
Why Use Fetch API?

Traditional web applications reload the entire page after every user action. This can be slow and interrupts the user experience.

With the Fetch API:

- Only the required data is exchanged.
- The page remains responsive.
- Network communication is asynchronous.
- Code is simpler and easier to maintain.

Fetch API Workflow



JavaScript Object

|

Update HTML

Fetch Syntax

```
fetch("students.php")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Understanding the Code

fetch("students.php")

Sends an HTTP GET request to the server.

response

Represents the HTTP response received from the server, including the status code, headers, and response body.

response.json()

Reads the response body and converts the JSON text into a JavaScript object.

.then()

Executes after the asynchronous operation completes successfully. Since network requests take time, JavaScript uses Promises to continue executing other code while waiting for the server's response.

.catch()

Handles errors such as network failures, invalid URLs, or server errors.

Complete Example

students.php

```
<?php
header("Content-Type: application/json");

$students = [
    ["name"=>"Alice","department"=>"CSE","cgpa"=>9.2],
    ["name"=>"Bob","department"=>"CSE","cgpa"=>8.9]
];

echo json_encode($students);
?>
```

index.html

```
<button onclick="loadStudents()">Load Students</button>

<div id="output"></div>

<script>
function loadStudents(){

    fetch("students.php")

    .then(response => response.json())

    .then(data => {

        let table = "<table border='1'>";
        table += "<tr><th>Name</th><th>Department</th><th>CGPA</th></tr>";

        data.forEach(student => {
            table += "<tr>";
            table += "<td>" + student.name + "</td>";
            table += "<td>" + student.department + "</td>";
            table += "<td>" + student.cgpa + "</td>";
            table += "</tr>";
        });

        table += "</table>";
```

```

        document.getElementById("output").innerHTML = table;

    })

    .catch(error => {
        document.getElementById("output").innerHTML =
            "Unable to retrieve student data.";
        console.error(error);
    });

}
</script>

```

Advantages of Fetch API

- Modern JavaScript standard.
- Promise-based asynchronous programming.
- Cleaner and more readable syntax.
- Supports GET, POST, PUT, DELETE, and other HTTP methods.
- Easy integration with RESTful APIs.
- Improved error handling.

AJAX vs Fetch API

AJAX (XMLHttpRequest)

Older technique

Callback-based

More verbose

Manual JSON parsing

More difficult to

maintain

Fetch API

Modern standard

Promise-based

Cleaner syntax

Convenient response.json()

Easier to read and maintain

Activity 5

Modify the application so that:

1. The PHP script returns details of five students.

2. The Fetch API retrieves the data.
3. The student information is displayed in an HTML table.
4. Display an appropriate error message if the server cannot be reached.

Challenge Exercise

Develop a **Student Information Viewer** with the following requirements:

1. Create an XML file containing details of five students.
2. Validate the XML using an XML Schema.
3. Write a PHP script that reads the XML file using `simplexml_load_file()`.
4. Convert the XML data to JSON using `json_encode()`.
5. Use the Fetch API to retrieve the JSON data.
6. Display the student information dynamically in an HTML table without refreshing the webpage.